What worked better with interfaces?

- Interfaces allow you a to have a bit more of a blueprint for each shape. When implementing the interfaces it requires you to meet the requirements for each shape to work.

What was more reusable or readable in the abstract class version?

- Having just one "blueprint" for each subclass makes the code a bit more readable/reusable.

Which design scales better?

- I honestly prefer the abstract class approach for this project, as interfaces seem a bit overkill for this simple of code, but I can see how it could be useful in bigger projects

Which approach would you choose for a real system and why?

- For a real system, depending on the project size I would probably go with Interfaces since if you are using more classes with multiple different features, some classes may require specific traits while others do not. With an abstract approach, you are limited to the class blueprint for all subclasses.

How did polymorphism change or benefit each design?

- I used a shared interface (Describable.java) to implement descriptions for each shape into both programs. This shared interface significantly shortens the amount of code/files, helping with comprehension.

Give an example where one approach was more maintainable or elegant

- As stated previously, using the abstract class for a project this simple made it a lot easier to format each subclass for what I needed, helping make sure I didn't miss anything that could cause an error.